

CSE-2202

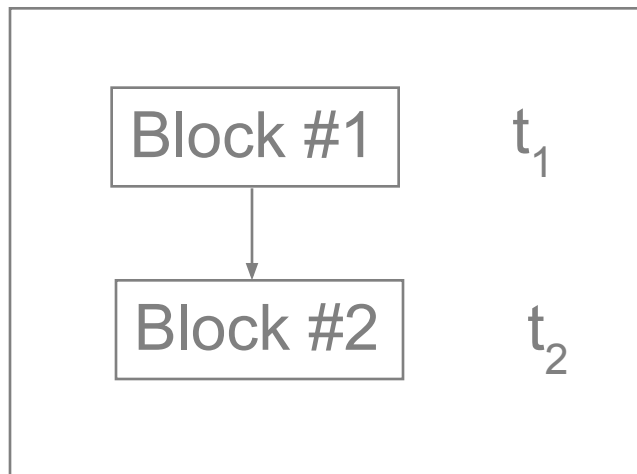
Design & Analysis of Algorithm

Recurrence Relation

GENERAL RULES FOR ANALYSIS(1/5)

1. Consecutive statements

- Maximum statement is the one counted
e.g. a fragment with single for-loop followed by double for-loop is $O(n^2)$.



$$t_1 + t_2 = \max(t_1, t_2)$$

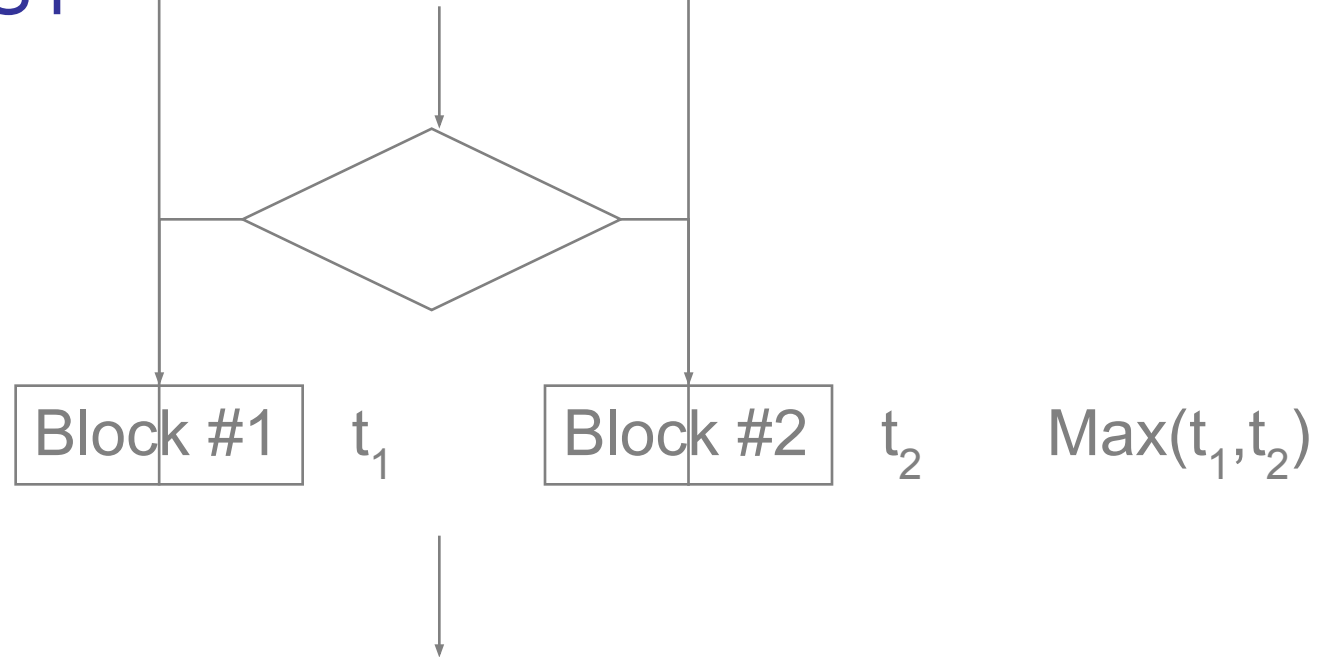
GENERAL RULES FOR ANALYSIS(2/5)

2. If/Else

if cond then S1

else

S2



GENERAL RULES FOR ANALYSIS(3/5)

3. For Loops

- Running time of a for-loop is at most the running time of the statements inside the for-loop times number of iterations

```
for (i = sum = 0; i < n; i++)  
    sum += a[i];
```

- for loop iterates n times, executes 2 assignment statements each iteration $\Rightarrow$ asymptotic complexity of $O(n)$

GENERAL RULES FOR ANALYSIS(4/5)

4. Nested For-Loops

- Analyze inside-out. Total running time is running time of the statement multiplied by product of the sizes of all the for-loops

e.g. `for (i =0; i < n; i++)`

`for (j = 0, sum = a[0]; j <= i ; j++)`

`sum += a[j];`

`printf("sum for subarray - through %d is %d\n", i, sum);`

GENERAL RULES FOR ANALYSIS(5/5)

$$\sum_{i=1}^n \sum_{j=1}^i \Theta(1) = \sum_{i=1}^n \Theta(i)$$

$$= \sum_{i=1}^n O(i)$$

$$= O\left[\sum_{i=1}^n i\right]$$

$$= O(n^2)$$

Recurrence Relations (1/2)

- A recurrence relation is an equation which is defined in terms of itself.
- Why are recurrences good things?
 - Many natural functions are easily expressed as recurrences:
 - $a_n = a_{n-1} + 1, a_1 = 1 \rightarrow a_n = n$ (polynomial)
 - $a_n = 2a_{n-1}, a_1 = 1 \rightarrow a_n = 2^n$ (exponential)
 - $a_n = na_{n-1}, a_1 = 1 \rightarrow a_n = n!$ (weird function)
- It is often easy to find a recurrence as the solution of a counting problem

Recurrence Relations (2/2)

- In both, we have general and boundary conditions, with the general condition breaking the problem into smaller and smaller pieces.
- The initial or boundary condition terminate the recursion.

Recurrence Equations

- A recurrence equation defines a function, say $T(n)$. The function is defined recursively, that is, the function $T(\cdot)$ appear in its definition. (recall recursive function call). The recurrence equation should have a base case.

For example:

$$T(n) = \begin{cases} T(n-1)+T(n-2), & \text{if } n > 1 \\ 1, & \text{if } n=1 \text{ or } n=0. \end{cases}$$

base case 

for convenience, we sometime write the recurrence equation as:

$$\begin{aligned} T(n) &= T(n-1)+T(n-2) \\ T(0) &= T(1) = 1. \end{aligned}$$

Recurrences

- The expression:

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

is a *recurrence*.

- Recurrence: an equation that describes a function in terms of its value on smaller functions

Recurrence Examples

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

More Recurrence equations:

$$T(n) = 2 * T(n/2) + 1,$$

$$T(1) = 1.$$

Base case;
initial condition.

$$T(n) = T(n-1) + n,$$

$$T(1) = 1.$$

Selection Sort

$$T(n) = 2 * T(n/2) + n,$$

$$T(1) = 1.$$

Merge Sort

Quick Sort

$$T(n) = 2 * T(n/2) + \log n,$$

$$T(1) = 1.$$

Heap Construction

$$T(n) = T(n/2) + 1,$$

$$T(1) = 0.$$

Binary search

Methods for Solving Recurrences

- Iteration method
- Substitution method
- Recursion tree method
- Master method

Simplifications:

- There are two simplifications we apply that won't affect asymptotic analysis
 - ignore floors and ceilings (justification in text)
 - assume base cases are constant, i.e., $T(n) = \Theta(1)$ for n small enough

Solving Recurrences: Iteration (convert to summation)

- Expand the recurrence
- Work some algebra to express as a summation
- Evaluate the summation

The Iteration Method

$$T(n) = c + T(n/2)$$

$$\begin{aligned} T(n) &= c + T(n/2) \\ &= c + c + T(n/4) \\ &= c + c + c + T(n/8) \end{aligned}$$

$$\begin{aligned} T(n/2) &= c + T(n/4) \\ T(n/4) &= c + T(n/8) \end{aligned}$$

Assume $n = 2^k$

$$\begin{aligned} T(n) &= \underbrace{c + c + \dots + c}_{k \text{ times}} + T(1) \\ &= c \lg n + T(1) \\ &= \Theta(\lg n) \end{aligned}$$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- $s(n) =$
 $c + s(n-1)$
 $c + c + s(n-2)$
 $2c + s(n-2)$
 $2c + c + s(n-3)$
 $3c + s(n-3)$
 ...
 $kc + s(n-k) = ck + s(n-k)$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have
 - $s(n) = ck + s(n-k)$
- What if $k = n$?
 - $s(n) = cn + s(0) = cn$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- So far for $n \geq k$ we have

- $s(n) = ck + s(n-k)$

- What if $k = n$?

- $s(n) = cn + s(0) = cn$

- So

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

- Thus in general

- $s(n) = cn$

Solving Recurrences: Iteration (convert to summation)

Example: $T(n) = 4T(n/2) + n$

$$\begin{aligned}T(n) &= 4T(n/2) + n && /**expand**/ \\&= 4(n/2 + 4T(n/4)) + n && /**simplify**/ \\&= 16T(n/4) + 2n + n && /**expand**/ \\&= 16(n/4 + 4T(n/8)) + 2n + n && /**simplify**/ \\&= 4^2\{4T(n/8) + n/4\} + (2+1)n \\&= 4^3T(n/2^3) + (2^2+2^1+2^0)n \\&= 4^kT(n/2^k) + (2^{k-1} + \dots + 2^1 + 2^0)n = 4^kT(n/2^k) + (2^k-1)n\end{aligned}$$

Assuming $2^k = n$,

$$T(n) = n^2T(1) + (n-1)n = n^2C + n^2 - n = \Theta(n^2)$$

The substitution method

1. Guess a solution
2. Use induction to prove that the solution works

Substitution method

- Guess a solution
 - $T(n) = O(g(n))$
 - Induction goal: **apply the definition of the asymptotic notation**
 - $T(n) \leq d g(n)$, for some $d > 0$ and $n \geq n_0$
 - Induction hypothesis: $T(k) \leq d g(k)$ for all $k < n$
- Prove the induction goal
 - Use the **induction hypothesis** to **find some values of the constants d and n_0** for which the **induction goal** holds

Example: Binary Search

$$T(n) = c + T(n/2)$$

- Guess: $T(n) = O(\lg n)$
 - Induction goal: $T(n) \leq d \lg n$, for some d and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq d \lg(n/2)$
- Proof of induction goal:

$$T(n) = T(n/2) + c \leq d \lg(n/2) + c$$

$$= d \lg n - d + c \leq d \lg n$$

$$\text{if: } -d + c \leq 0, d \geq c$$

Example 2

$$T(n) = T(n-1) + n$$

- Guess: $T(n) = O(n^2)$
 - Induction goal: $T(n) \leq c n^2$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(k-1) \leq c(k-1)^2$ for all $k < n$

- Proof of induction goal:

$$T(n) = T(n-1) + n \leq c(n-1)^2 + n$$

$$= cn^2 - (2cn - c - n) \leq cn^2$$

$$\text{if: } 2cn - c - n \geq 0 \Leftrightarrow c \geq n/(2n-1) \Leftrightarrow c \geq 1/(2 - 1/n)$$

- For $n \geq 1 \Rightarrow 2 - 1/n \geq 1 \Rightarrow$ any $c \geq 1$ will work

Example 3

$$T(n) = 2T(n/2) + n$$

- Guess: $T(n) = O(n \lg n)$
 - Induction goal: $T(n) \leq cn \lg n$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq cn/2 \lg(n/2)$
- Proof of induction goal:

$$\begin{aligned} T(n) &= 2T(n/2) + n \leq 2c (n/2) \lg(n/2) + n \\ &= cn \lg n - cn + n \leq cn \lg n \end{aligned}$$

$$\text{if: } -cn + n \leq 0 \Rightarrow c \geq 1$$

Changing variables

$$T(n) = 2T(\sqrt{n}) + \lg n$$

– Rename: $m = \lg n \Rightarrow n = 2^m$

$$T(2^m) = 2T(2^{m/2}) + m$$

– Rename: $S(m) = T(2^m)$

$$S(m) = 2S(m/2) + m \Rightarrow S(m) = O(m \lg m)$$

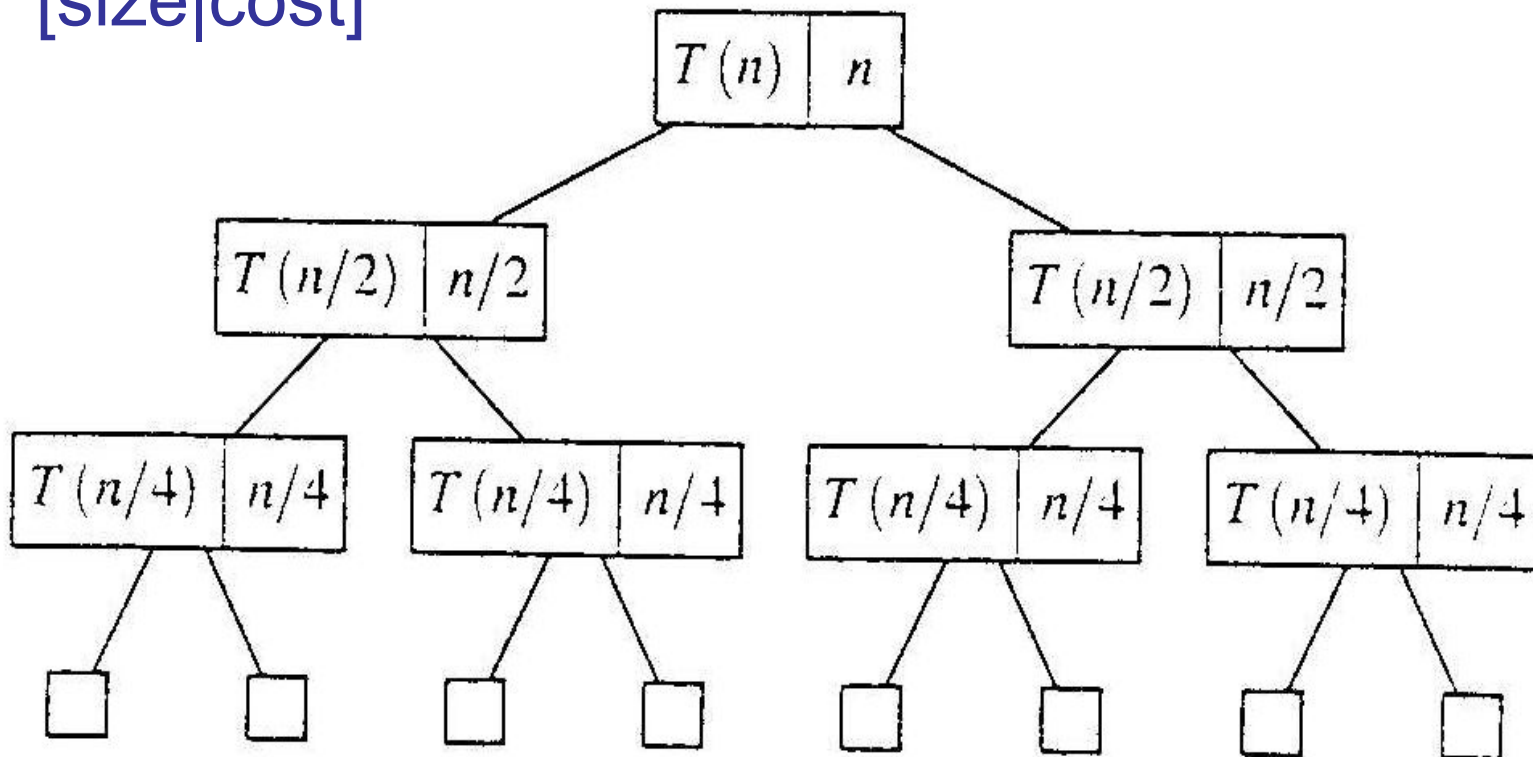
(demonstrated before)

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$$

Idea: transform the recurrence to one that you have seen before

Evaluate recursive equation using Recursion Tree

- Evaluate: $T(n) = T(n/2) + T(n/2) + n$
 - Work copy: $T(k) = T(k/2) + T(k/2) + k$
 - For $k=n/2$, $T(n/2) = T(n/4) + T(n/4) + (n/2)$
- [size|cost]



Recursion-tree method

- A recursion tree models the costs (time) of a recursive execution of an algorithm.
- The recursion tree method is **good for generating guesses** for the substitution method.
- The recursion-tree method can be **unreliable**.
- The recursion-tree method promotes intuition, however.

Recursion Tree e.g.

- To evaluate the total cost of the recursion tree
 - sum all the non-recursive costs of all nodes
 - = Sum (rowSum(cost of all nodes at the same depth))
- Determine the maximum depth of the recursion tree:
 - For our example, at tree depth d the size parameter is $n/(2^d)$
 - the size parameter converging to base case, i.e. case 1
 - such that, $n/(2^d) = 1$,
 - $d = \lg(n)$
 - The rowSum for each row is n
- Therefore, the total cost, $T(n) = n \lg(n)$

Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

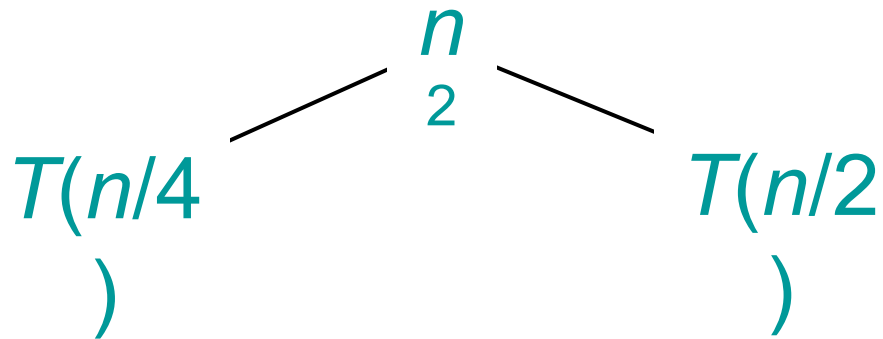
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

$T(n)$
)

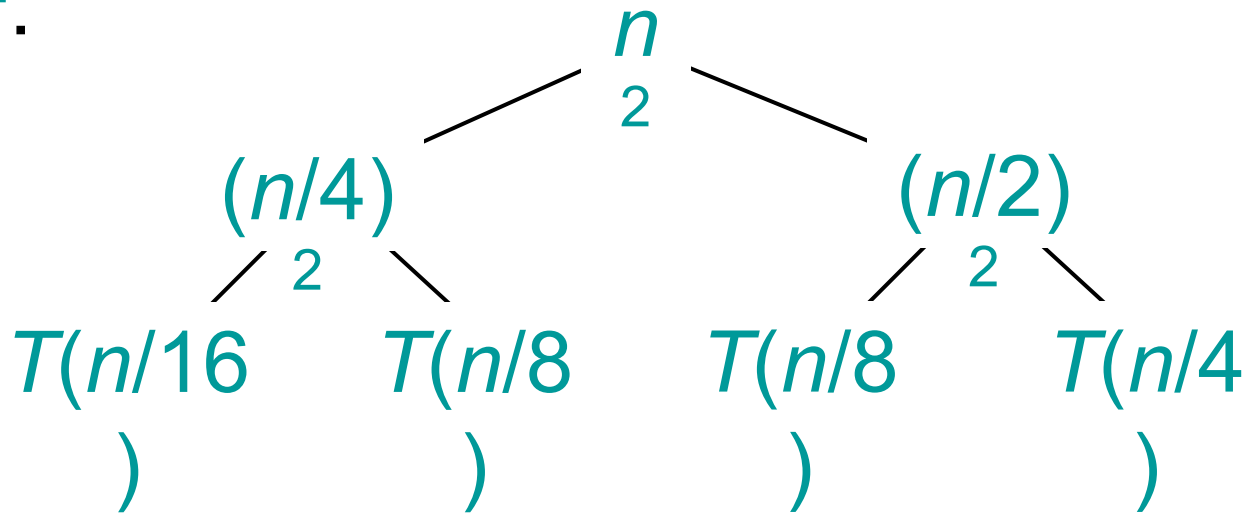
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



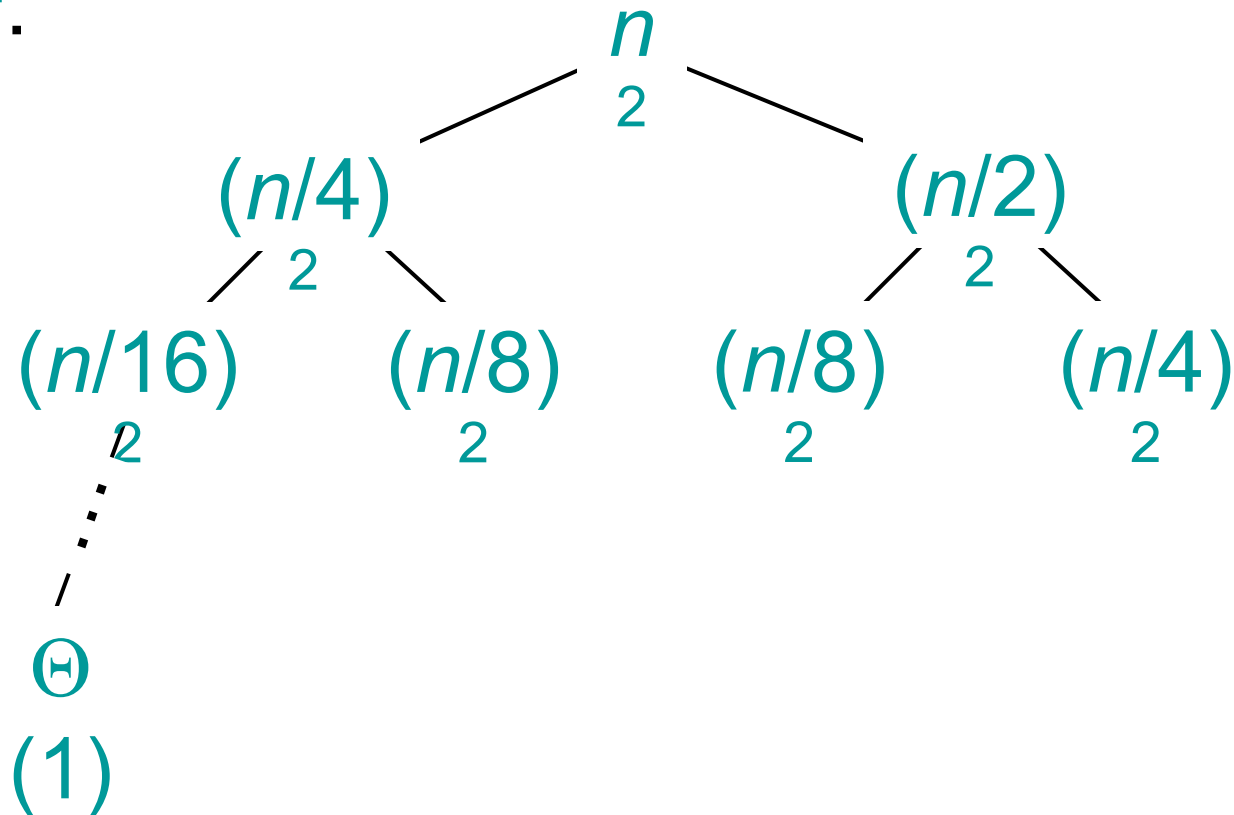
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



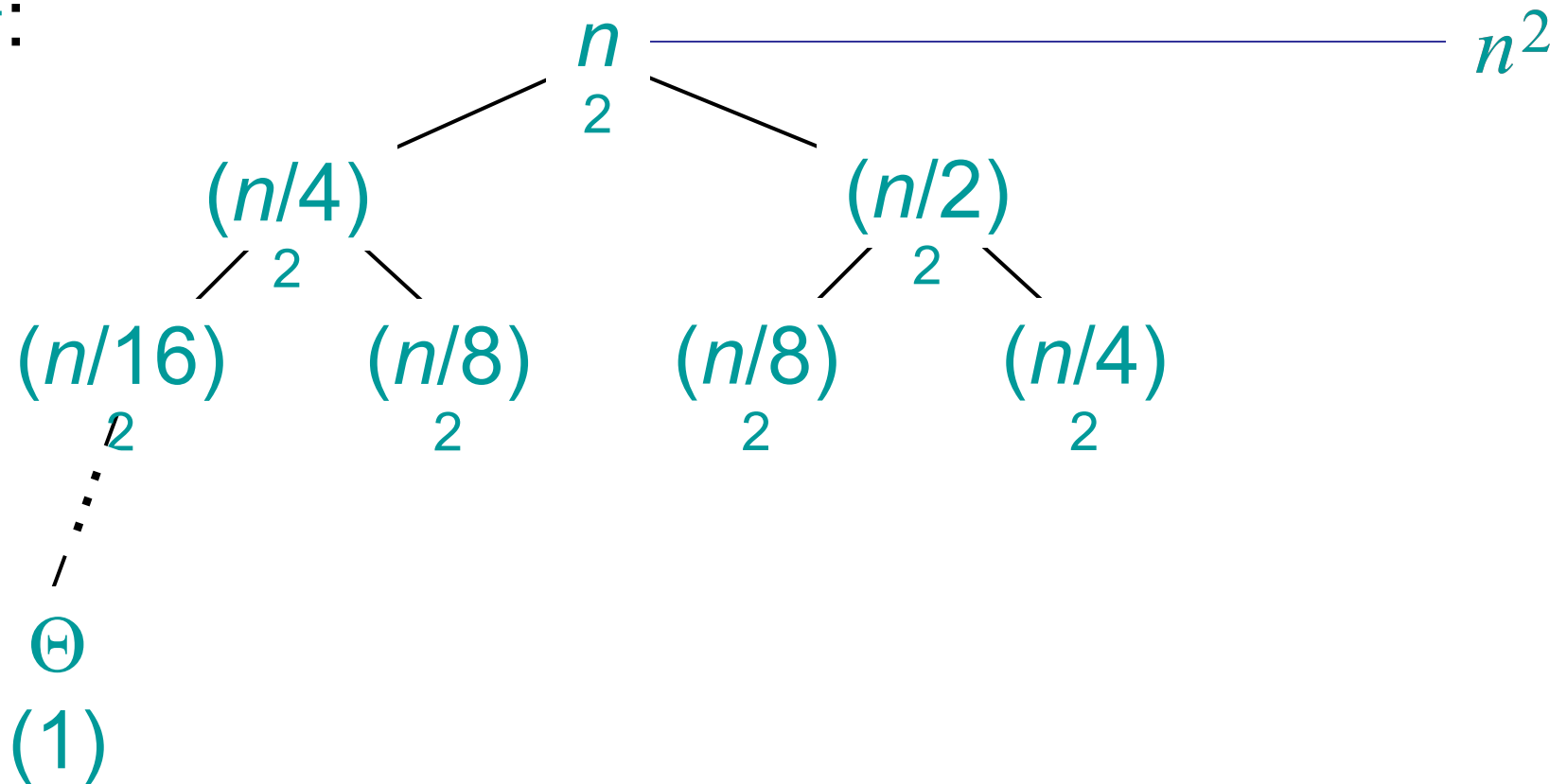
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



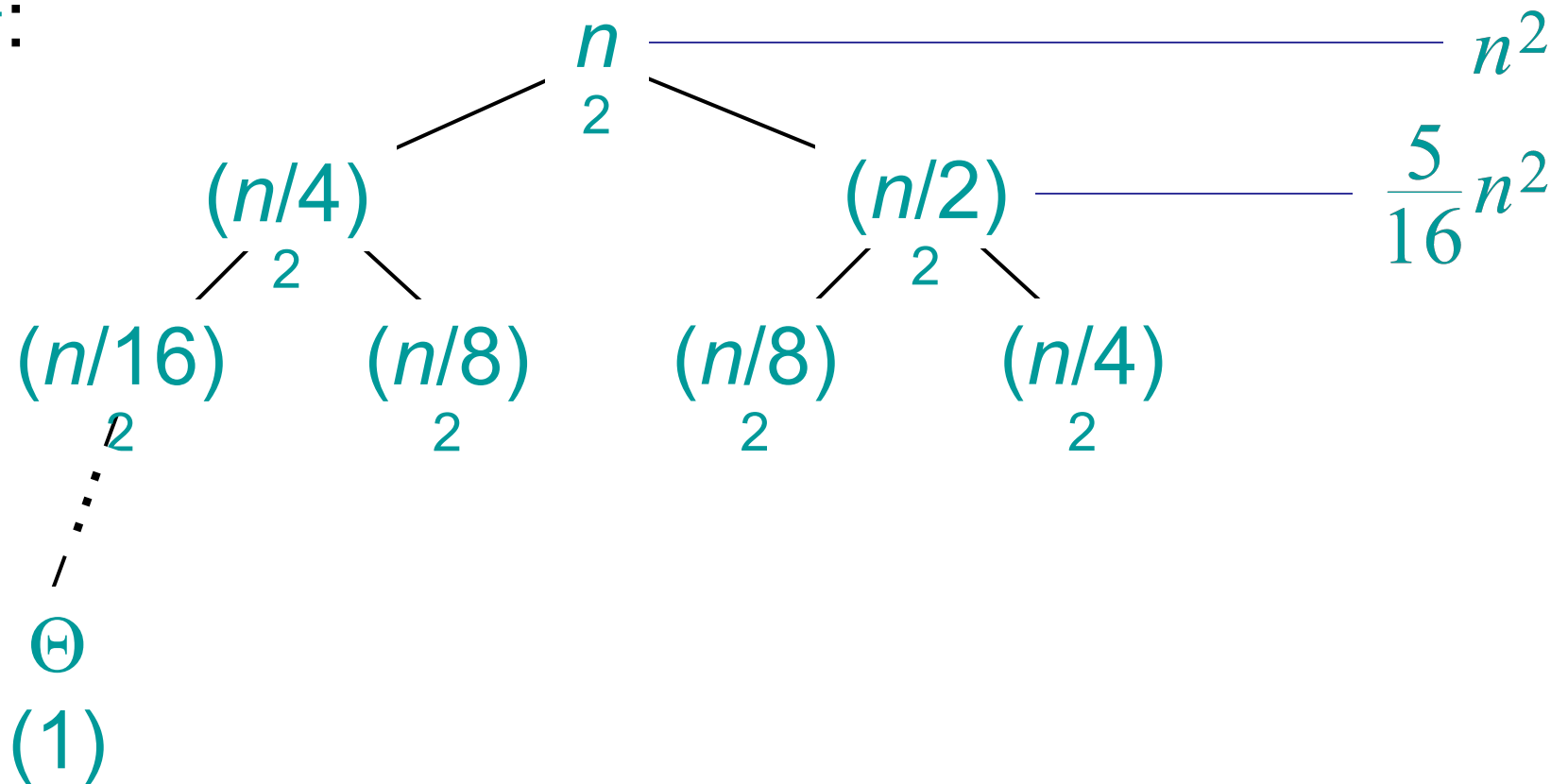
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



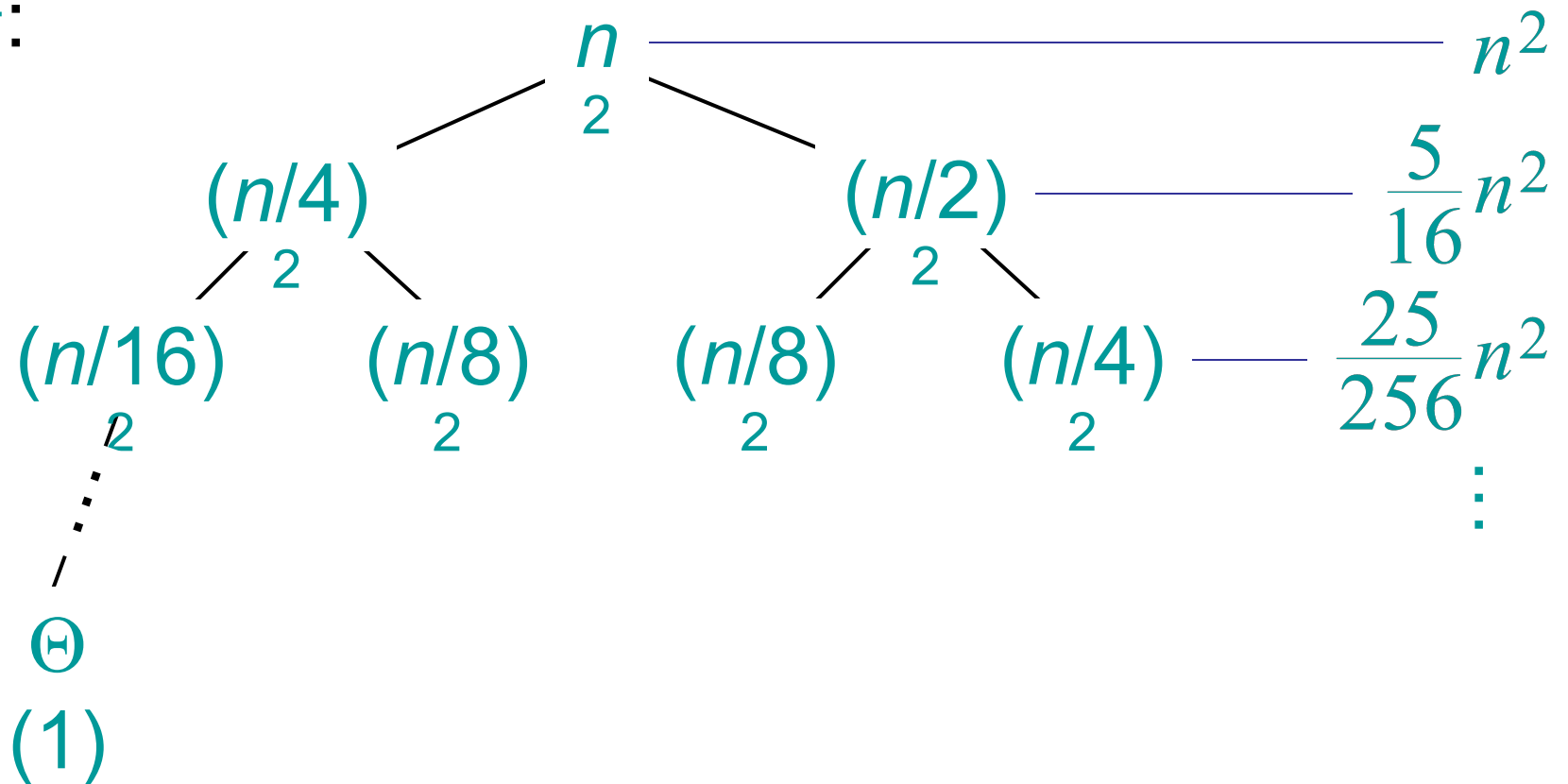
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



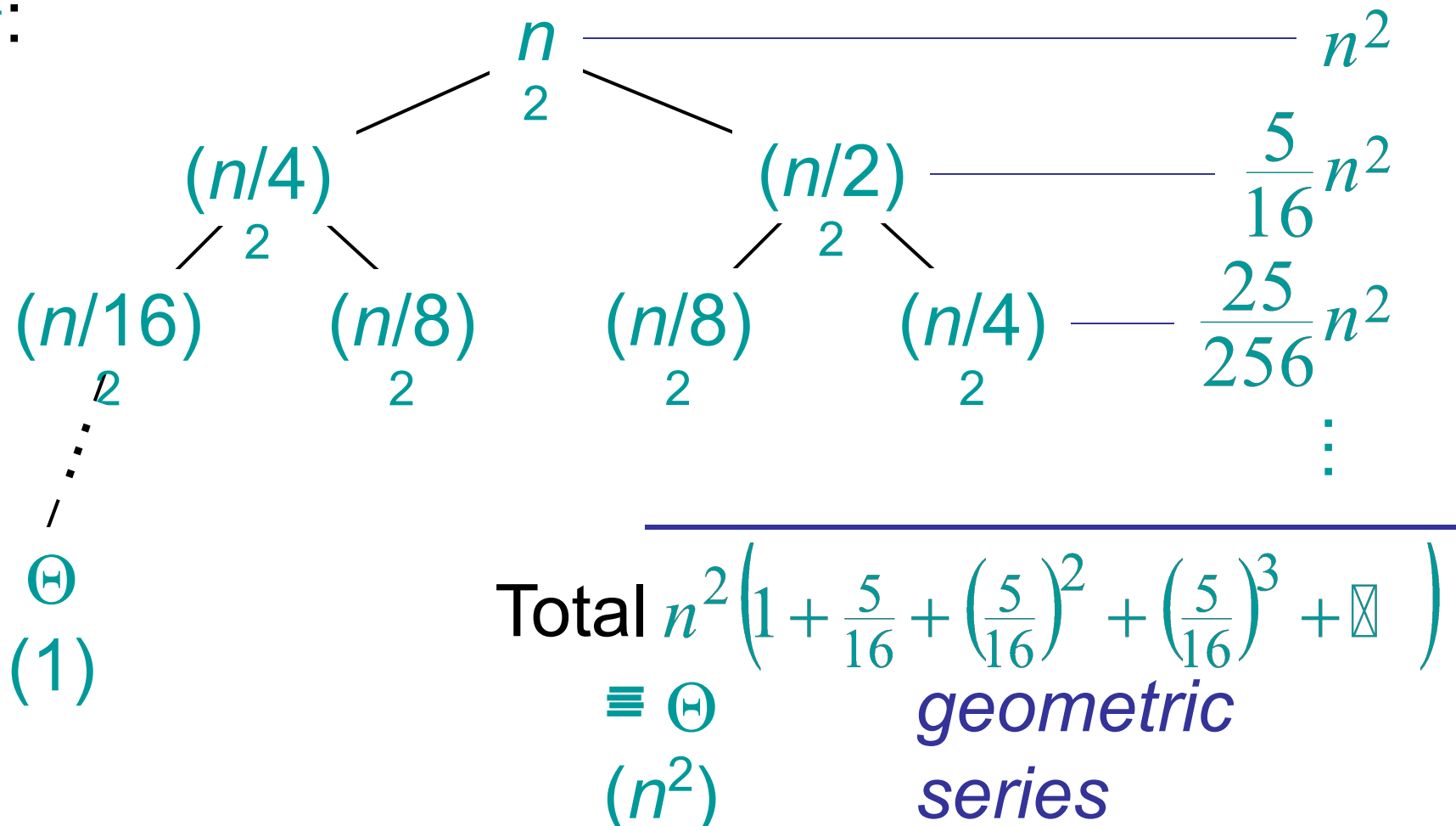
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



The Master Method

- Based on the **Master theorem**.
- “**Cookbook**” approach for solving recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

- $a \geq 1, b > 1$ are constants.
 - $f(n)$ is asymptotically positive.
 - n/b may not be an integer, but we ignore floors and ceilings.
- Requires memorization of three cases.

The Master Theorem

Theorem 4.1

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and Let $T(n)$ be defined on nonnegative integers by the recurrence $T(n) = aT(n/b) + f(n)$, where we can replace n/b by $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. $T(n)$ can be bounded asymptotically in three cases:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,
and if, for some constant $c < 1$ and all sufficiently large n ,
we have $a \cdot f(n/b) \leq c f(n)$, then $T(n) = \Theta(f(n))$.

Master Method – Examples

- $T(n) = 16T(n/4) + n$
 - $a = 16, b = 4, n^{\log_b a} = n^{\log_4 16} = n^2$.
 - $f(n) = n = O(n^{\log_b a - \varepsilon}) = O(n^{2-\varepsilon})$, where $\varepsilon = 1 \Rightarrow$ **Case 1**.
 - Hence, $T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$.
- $T(n) = T(3n/7) + 1$
 - $a = 1, b = 7/3$, and $n^{\log_b a} = n^{\log_{7/3} 1} = n^0 = 1$
 - $f(n) = 1 = \Theta(n^{\log_b a}) \Rightarrow$ **Case 2**.
 - Therefore, $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(\lg n)$

Master Method – Examples

- $T(n) = 3T(n/4) + n \lg n$
 - $a = 3, b=4$, thus $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$
 - $f(n) = n \lg n = \Omega(n^{\log_4 3 + \varepsilon})$ where $\varepsilon \approx 0.2 \Rightarrow$ **Case 3**.
 - Therefore, $T(n) = \Theta(f(n)) = \Theta(n \lg n)$.
- $T(n) = 2T(n/2) + n \lg n$
 - $a = 2, b=2$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\log_2 2} = n$
 - $f(n)$ is asymptotically larger than $n^{\log_b a}$, but **not polynomially larger**. The ratio $\lg n$ is asymptotically less than n^ε for any positive ε . Thus, the Master Theorem **doesn't** apply here.